AnimLOG

Cómo funciona, explicado fácil

Para quién es este documento

Para cualquiera que quiera entender cómo funciona esta aplicación, sin saber programar. No hay que leer una sola línea de código: todo se explica con ejemplos de la vida real.

Trabajo Práctico Full Stack — FastAPI + SQLite + JavaScript

1. Primero lo primero: qué es esto

AnimLOG es una libreta de anime. Una pantalla donde anotás qué animes viste: el título, el estudio que lo hizo, el año, el género, cuántos capítulos tiene y si ya lo viste o no.

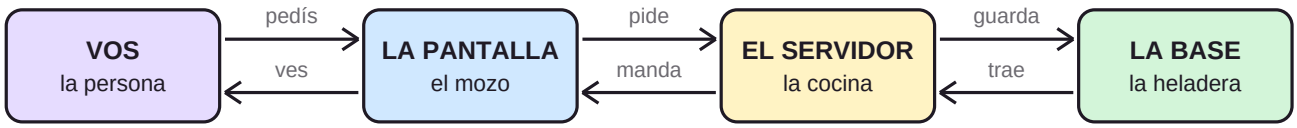
Podés **agregar** uno nuevo, **ver** la lista completa, **editar** alguno que cargaste mal y **borrar** el que no te interesa más. Esas cuatro cosas juntas son lo que en programación se llama un **CRUD** (Crear, Leer, Actualizar y Borrar).

> **La sorpresa**
>
> Aunque lo veas como una sola página, en realidad son **dos programas distintos** corriendo al mismo tiempo en tu computadora, hablándose entre ellos. Entender eso es entender el 80% del proyecto.

2. La idea clave: el restaurante

La mejor forma de entender esto es pensarlo como un restaurante. Fijate quién es quién:

En el restaurante	En AnimLOG	Qué hace
Vos, el comensal	La persona usando la app	Pide cosas y espera que lleguen
El mozo y la carta	La **pantalla** (frontend)	Es lo único que ves. Toma tu pedido y te trae el resultado
La cocina	El **servidor** (backend)	Hace el trabajo de verdad. Vos no la ves nunca
La heladera	La **base de datos**	Donde quedan guardadas las cosas
La comanda escrita	El **JSON**	El papelito con el pedido anotado de forma ordenada
El pasillo a la cocina	El **HTTP**	El camino por donde van y vienen los pedidos



Vos solo hablás con el mozo. Nunca entrás a la cocina.

> **Por qué no entrás a la cocina**
>
> Porque sería un desastre: cualquiera podría agarrar lo que quiera de la heladera o romper algo. Lo mismo pasa acá: **la pantalla nunca toca la base de datos directamente.** Siempre se lo pide al servidor, que es el único que tiene permiso de abrirla. Así, si alguien trucha la pantalla, igual no puede hacer lo que quiera con los datos.

3. Qué hace cada archivo

El proyecto tiene dos carpetas. Una es el mozo y la otra es la cocina.

La carpeta frontend (el mozo)

Archivo	Su trabajo	En el restaurante sería...
index.html	Los cuadraditos donde escribís y la tabla donde se ve todo	La carta y el formulario del pedido
style.css	Los colores, el rosa, los bordes redondeados	La decoración del salón
api.js	Lleva y trae los pedidos al servidor	Las piernas del mozo: el único que camina hasta la cocina
ui.js	Dibuja la tabla y los carteles en la pantalla	La mano del mozo que apoya el plato en la mesa
main.js	Escucha los clics y le dice a los otros dos qué hacer	La cabeza del mozo, que coordina todo

La carpeta backend (la cocina)

Archivo	Su trabajo	En el restaurante sería...
main.py	Recibe los pedidos que llegan y reparte el trabajo	El jefe de cocina que lee la comanda
models.py	Revisa que el pedido tenga sentido antes de cocinarlo	El que dice 'che, esto está mal pedido'
managers.py	Guarda y saca cosas de la base de datos	El cocinero que abre la heladera
database.py	Abre y cierra la heladera de forma segura	La llave de la heladera
animes.db	El archivo donde quedan guardados los animes	La heladera en sí

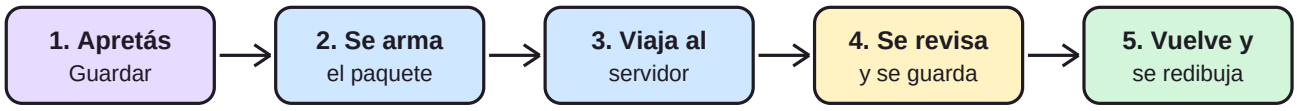
Por qué tantos archivos y no uno solo

Porque cuando algo se rompe, sabés exactamente dónde mirar. Si la tabla se ve fea, el problema está en **ui.js**. Si no se guarda nada, está en **managers.py**. Si fuera todo un archivo gigante, habría que leerlo entero cada vez. Es lo mismo que tener la ropa en cajones separados en vez de todo tirado en una pila.

4. Qué pasa cuando apretás "Guardar"

Esta es la parte importante. Parece que apretás un botón y aparece la fila en la tabla, pero en el medio pasan 9 cosas. Seguilas con la analogía del restaurante al lado:

1. **Apretás Guardar.** El archivo *main.js* se da cuenta. — *Levantás la mano y llamás al mozo.*
2. **Lee lo que escribiste** en cada cuadradito y lo arma en un paquetito ordenado (eso es el JSON). — *El mozo anota tu pedido en la comanda.*
3. **Aparece el cartel "Cargando..."** — *El mozo te dice "ya se lo traigo" para que no te quedes mirando la nada.*
4. **El paquetito viaja** al servidor. — *El mozo camina hasta la cocina con la comanda.*
5. **El servidor revisa que el pedido tenga sentido.** Si pusiste el año 1200 o 0 capítulos, lo rechaza acá mismo. — *El cocinero lee la comanda y dice "esto no se puede hacer".*
6. **Si está todo bien, lo guarda** en la base de datos y le pone un número (el id). — *Cocina el plato y lo guarda en la heladera con una etiqueta.*
7. **Devuelve el anime ya guardado**, ahora con su número. — *El plato sale de la cocina.*
8. **La pantalla limpia el formulario** y pide la lista completa de nuevo. — *El mozo levanta los platos viejos y trae la mesa servida de cero.*
9. **Se dibuja la tabla actualizada** y se apaga el "Cargando...". — *Ya podés comer.*



Todo esto pasa en menos de un parpadeo.

El detalle que sorprende a todos

La tabla no se actualiza sola. Después de guardar, la pantalla *vuelve a pedir la lista entera* y la dibuja de nuevo desde cero. Es como si el mozo, cada vez que trae un plato, levantara toda la mesa y la volviera a poner. Suena ineficiente, pero garantiza que lo que ves es exactamente lo que hay guardado, sin sorpresas.

5. El portero del edificio (esto se llama CORS)

Acá hay algo que confunde a todo el mundo la primera vez. Los dos programas corren en tu propia computadora, pero en **puertas distintas**: la pantalla en la puerta **5500** y el servidor en la **8000**.

Para el navegador, dos puertas distintas son **dos lugares distintos**, aunque sea el mismo edificio. Y el navegador tiene una regla de seguridad muy estricta:

La regla del navegador

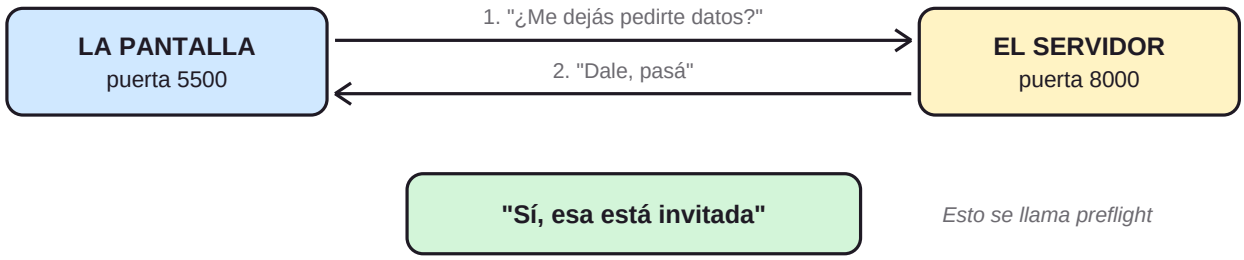
"Una página solo puede leer respuestas del **mismo lugar** de donde vino. Si pide algo a otro lado, no la dejo leer la respuesta."

¿Por qué esa regla tan molesta?

Imaginate que entrás a una página cualquiera y esa página, por atrás, le pide tus datos al homebanking donde estás logueado y se los manda al que la hizo. Sin esta regla, eso sería posible. **La regla existe para protegerte.**

Entonces, ¿cómo lo arreglamos?

Con una **lista de invitados**. El servidor avisa: "a la pantalla del puerto 5500 la conozco, dejala pasar". Eso es **CORS**. No se apaga la seguridad: se le dice al portero quién está invitado.



Ojo con esto

Si abrís el archivo **index.html** haciendo doble clic, no funciona nada. Para el navegador eso no viene de ninguna puerta, así que el portero lo rechaza igual. Hay que abrirlo con **Live Server** para que quede en la puerta 5500, que es la que está en la lista de invitados.

6. El filtro: por qué a veces sale un cartel rojo

Antes de guardar nada, el servidor revisa que lo que mandaste tenga sentido. Es como el cocinero que lee la comanda y dice "pará, esto está mal".

Campo	Regla	Por qué
Título	No puede estar vacío	Un anime sin nombre no sirve de nada
Estudio	No puede estar vacío	Ídem
Año	Entre 1900 y 2100	No existe un anime del año 1200
Episodios	Por lo menos 1	Un anime de 0 capítulos no existe
Ya lo vi	Es opcional	Si no lo marcás, se guarda como 'no visto'

Si algo no cumple, el servidor **no guarda nada** y avisa que está mal. La pantalla agarra ese aviso, lo traduce a algo entendible y lo muestra en el cartel rojo. Lo importante: **te dice todos los errores juntos**, no de a uno, así no tenés que adivinar de a poco.

Los dos carteles rojos distintos

"Revisá estos campos..." significa que escribiste algo mal: corregilo y listo.

"No se pudo conectar con la API" significa otra cosa: lo que escribiste puede estar perfecto, pero **la cocina está cerrada**. Te olvidaste de prender el servidor.

7. La cocina tiene amnesia

Algo raro pero importante: **el servidor no se acuerda de nada entre un pedido y el siguiente.** Cada vez que le pedís algo, es como si fuera la primera vez que te ve.

Si le pedís la lista y después le decís "borrá el 3", el servidor no tiene idea de que sos la misma persona. Por eso cada pedido tiene que traer **toda** la información: por eso se le dice "borrá el **3**" y no simplemente "borrá ese".

Lo bueno de la amnesia

Si la app se llenara de gente, podrías poner tres cocinas iguales en vez de una. Como ninguna se acuerda de nada, **da lo mismo cuál te atienda**: el resultado es idéntico. Y si una se rompe, las otras dos siguen trabajando y nadie se entera. Eso es mucho más difícil de lograr si la cocina tuviera que acordarse de cada cliente.

8. Diccionario de palabras raras

Palabra	Qué significa, en criollo
Frontend	Todo lo que ves y tocás: la pantalla. El mozo.
Backend	Lo que no ves y hace el trabajo pesado. La cocina.
API	El mostrador por donde la cocina recibe pedidos y entrega platos.
Endpoint	Cada cosa puntual que podés pedir. Como cada plato de la carta.
HTTP	El idioma y el camino por donde viajan los pedidos.
JSON	El formato ordenado en que se escribe el pedido. La comanda.
CRUD	Las 4 cosas básicas: Crear, Leer, Actualizar y Borrar.
Base de datos	Donde quedan guardadas las cosas. La heladera.
SQLite	El tipo de heladera que usamos: un solo archivo, sin instalar nada.
CORS	La lista de invitados que deja pasar a la pantalla.
Servidor	El programa que está prendido esperando pedidos.
Puerto	El número de la puerta donde cada programa espera. 5500 y 8000.
Validación	El control de que lo que mandaste tenga sentido.
Error 422	"Me mandaste algo mal". Lo arreglás vos corrigiendo el dato.
Error 404	"Eso que pedís no existe".
Error 500	"Me rompí yo". No es tu culpa, es del servidor.

9. Cómo probarlo vos mismo

Tres pasos, en este orden. **El orden importa:** si prendés la pantalla sin la cocina, te va a salir el cartel rojo.

1. **Prender la cocina.** Abrí una terminal en la carpeta *backend* y escribí: **uvicorn main:app --reload**. Cuando diga "Application startup complete", está lista.
2. **Prender la pantalla.** En VS Code, clic derecho sobre *frontend/index.html* y elegí **Open with Live Server**. Se abre sola en el navegador.
3. **Usarla.** Cargá un anime, editalo, borralo. Si algo sale mal, mirá el cartel rojo: te dice si el problema es lo que escribiste o que la cocina está apagada.

Bonus: la cocina tiene su propio menú

Con el servidor prendido, entrá a **http://127.0.0.1:8000/docs** desde el navegador. Vas a ver una página que lista todo lo que la cocina sabe hacer, y podés probar cada cosa desde ahí, sin la pantalla. Esa página la genera sola FastAPI: nadie la escribió.

Y eso es todo. Si entendiste lo del mozo, la cocina y la heladera, entendiste cómo funciona cualquier aplicación web que uses todos los días.